

VALID STACK POP SEQUENCE:

```
import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] push = new int[n];
        int[] pop = new int[n];
        for (int i = 0; i < n; i++)
        {
            push[i] = sc.nextInt();
        }
        for (int i = 0; i < n; i++)
        {
            pop[i] = sc.nextInt();
        }
        Stack<Integer> stack = new Stack<>();
        int j = 0;
        for (int i = 0; i < n; i++)
        {
            stack.push(push[i]);
            while (!stack.isEmpty() && j < n && stack.peek() == pop[j])
            {
                stack.pop();
                j++;
            }
        }
        System.out.println(stack.isEmpty());
    }
}
```

OUTPUT:

```
5
1 2 3 4 5
4 5 3 2 1
true
```

QUEUE:**QUEUE METHODS:**

```
import java.util.*;
public class Main
{
```

```

public static void main(String[] args)
{
    Queue<String> queue = new LinkedList<>();
    queue.offer("Karina");
    queue.offer("Winter");
    queue.offer("Ananya");
    System.out.println("Queue: " + queue);
    queue.add("Jisoo");
    System.out.println("After add(): " + queue);
    System.out.println("Peek: " + queue.peek());
    System.out.println("Element: " + queue.element());
    System.out.println("Contains Winter? " + queue.contains("Winter"));
    System.out.println("Size: " + queue.size());
    System.out.println("Poll: " + queue.poll());
    System.out.println("After poll(): " + queue);
    System.out.println("Remove: " + queue.remove());
    System.out.println("After remove(): " + queue);
    System.out.println("Is Empty? " + queue.isEmpty());
    queue.clear();
    System.out.println("After clear(): " + queue);
}
}

```

OUTPUT:

```

Queue: [Karina, Winter, Ananya]
After add(): [Karina, Winter, Ananya, Jisoo]
Peek: Karina Element: Karina
Contains Winter? true
Size: 4
Poll: Karina
After poll(): [Winter, Ananya, Jisoo]
Remove: Winter After remove(): [Ananya, Jisoo]
Is Empty? false
After clear(): []

```

ARRAYDEQUE:

```

import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Deque<String> deque = new ArrayDeque<>();
        deque.addFirst("Cha Eun-woo");
        System.out.println("After addFirst(): " + deque);
    }
}

```

```

deque.addLast("Sana");
System.out.println("After addLast(): " + deque);
deque.offerFirst("Rosé");
System.out.println("After offerFirst(): " + deque);
deque.offerLast("Hyunjin");
System.out.println("After offerLast(): " + deque);
System.out.println("First Element: " + deque.getFirst());
System.out.println("Last Element: " + deque.getLast());
System.out.println("Peek First: " + deque.peekFirst());
System.out.println("Peek Last: " + deque.peekLast());
System.out.println("Removed First: " + deque.removeFirst());
System.out.println("After removeFirst(): " + deque);
System.out.println("Removed Last: " + deque.removeLast());
System.out.println("After removeLast(): " + deque);
System.out.println("Poll First: " + deque.pollFirst());
System.out.println("After pollFirst(): " + deque);
deque.addLast("IU");
deque.addLast("Felix");
System.out.println("After adding elements: " + deque);
System.out.println("Poll Last: " + deque.pollLast());
System.out.println("After pollLast(): " + deque);
}
}

```

OUTPUT:

```

After addFirst(): [Cha Eun-woo]
After addLast(): [Cha Eun-woo, Sana]
After offerFirst(): [Rosé, Cha Eun-woo, Sana]
After offerLast(): [Rosé, Cha Eun-woo, Sana, Hyunjin]
First Element: Rosé
Last Element: Hyunjin
Peek First: Rosé
Peek Last: Hyunjin
Removed First: Rosé
After removeFirst(): [Cha Eun-woo, Sana, Hyunjin]
Removed Last: Hyunjin
After removeLast(): [Cha Eun-woo, Sana]
Poll First: Cha Eun-woo
After pollFirst(): [Sana]
After adding elements: [Sana, IU, Felix]
Poll Last: Felix
After pollLast(): [Sana, IU]

```

PRIORITYQUEUE:

MIN HEAP:

```
import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Queue<String> queue = new PriorityQueue<>();
        queue.offer("Karina");
        queue.offer("Winter");
        queue.offer("Ananya");
        System.out.println("Queue: " + queue);
        queue.add("Jisoo");
        System.out.println("After add(): " + queue);
        System.out.println("Peek: " + queue.peek());
        System.out.println("Element: " + queue.element());
        System.out.println("Contains Winter? " + queue.contains("Winter"));
        System.out.println("Size: " + queue.size());
        System.out.println("Poll: " + queue.poll());
        System.out.println("After poll(): " + queue);
        System.out.println("Remove: " + queue.remove());
        System.out.println("After remove(): " + queue);
        System.out.println("Is Empty? " + queue.isEmpty());
        queue.clear();
        System.out.println("After clear(): " + queue);
    }
}
```

OUTPUT:

```
Queue: [Ananya, Winter, Karina]
After add(): [Ananya, Jisoo, Karina, Winter]
Peek: Ananya
Element: Ananya
Contains Winter? true
Size: 4
Poll: Ananya
After poll(): [Jisoo, Winter, Karina]
Remove: Jisoo
After remove(): [Karina, Winter]
Is Empty? false
After clear(): []
```

MAX HEAP:

```
import java.util.*;
```

```

public class Main
{
    public static void main(String[] args)
    {
        Queue<String> queue = new PriorityQueue<>( Collections.reverseOrder());
        queue.offer("Karina");
        queue.offer("Winter");
        queue.offer("Ananya");
        System.out.println("Queue: " + queue);
        queue.add("Jisoo");
        System.out.println("After add(): " + queue);
        System.out.println("Peek: " + queue.peek());
        System.out.println("Element: " + queue.element());
        System.out.println("Contains Winter? " + queue.contains("Winter"));
        System.out.println("Size: " + queue.size());
        System.out.println("Poll: " + queue.poll());
        System.out.println("After poll(): " + queue);
        System.out.println("Remove: " + queue.remove());
        System.out.println("After remove(): " + queue);
        System.out.println("Is Empty? " + queue.isEmpty());
        queue.clear();
        System.out.println("After clear(): " + queue);
    }
}

```

OUTPUT:

```

Queue: [Winter, Karina, Ananya]
After add(): [Winter, Karina, Ananya, Jisoo]
Peek: Winter
Element: Winter
Contains Winter? true
Size: 4
Poll: Winter
After poll(): [Karina, Jisoo, Ananya]
Remove: Karina
After remove(): [Jisoo, Ananya]
Is Empty? false
After clear(): []

```

```

import java.util.*;
public class Main
{
    public static void main(String[] args)
    {

```

```

Scanner sc = new Scanner(System.in);
Queue<Integer> queue = new LinkedList<>();
Stack<Integer> stack = new Stack<>();
System.out.print("Enter the number of elements: ");
int n = sc.nextInt();
System.out.println("Enter the queue elements:");
for (int i = 0; i < n; i++)
{
    queue.offer(sc.nextInt());
}
System.out.print("Enter k: ");
int k = sc.nextInt();
if (k > n || k < 0)
{
    System.out.println("Invalid value of k!");
    return;
}
System.out.println("Original Queue: " + queue);
for (int i = 0; i < k; i++)
{
    stack.push(queue.poll());
}
while (!stack.isEmpty())
{
    queue.offer(stack.pop());
}
    for (int i = 0; i < n - k; i++)
    {
        queue.offer(queue.poll());
    }
System.out.println("Queue after reversing first " + k + " elements: " + queue);
}
}

```

OUTPUT:

```

Enter the number of elements: 5
Enter the queue elements:
95 28 32 12 20
Enter k: 3
Original Queue: [95, 28, 32, 12, 20]
Queue after reversing first 3 elements: [32, 28, 95, 12, 20]

```

SET:

HASHSET:

```

import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        HashSet<String> set = new HashSet<>();
        set.add("Rosé");
        set.add("Cha Eun-woo");
        set.add("Sana");
        set.add("Rosé");
        set.add("Felix");
        set.add(null);
        set.add(null);
        System.out.println("After add(): " + set);
        System.out.println("Contains Sana? " + set.contains("Sana"));
        System.out.println("Size: " + set.size());
        set.remove("Felix");
        System.out.println("After remove(): " + set);
        System.out.println("Is Empty? " + set.isEmpty());
        set.clear();
        System.out.println("After clear(): " + set);
    }
}

```

OUTPUT:

```

After add(): [null, Rosé, Felix, Sana, Cha Eun-woo]
Contains Sana? true
Size: 5
After remove(): [null, Rosé, Sana, Cha Eun-woo]
Is Empty? false
After clear(): []

```

LINKEDHASHSET:

```

import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        LinkedHashSet<String> set = new LinkedHashSet<>();
        set.add("Rosé");
        set.add("Cha Eun-woo");
        set.add("Sana");
        set.add("Rosé");
        set.add("Felix");
    }
}

```

```

        set.add(null);
        set.add(null);
        System.out.println("After add(): " + set);
        System.out.println("Contains Sana? " + set.contains("Sana"));
        System.out.println("Size: " + set.size());
        set.remove("Felix");
        System.out.println("After remove(): " + set);
        System.out.println("Is Empty? " + set.isEmpty());
        set.clear();
        System.out.println("After clear(): " + set);
    }
}

```

OUTPUT:

After add(): [Rosé, Cha Eun-woo, Sana, Felix, null]
 Contains Sana? true
 Size: 5
 After remove(): [Rosé, Cha Eun-woo, Sana, null]
 Is Empty? false
 After clear(): []

TREESET:

```

import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        TreeSet<Integer> set = new TreeSet<>();
        set.add(30);
        set.add(10);
        set.add(50);
        set.add(20);
        set.add(40);
        set.add(30);
        System.out.println("After add(): " + set);
        System.out.println("Contains 20? " + set.contains(20));
        System.out.println("Size: " + set.size());
        System.out.println("First Element: " + set.first());
        System.out.println("Last Element: " + set.last());
        System.out.println("Floor of 35: " + set.floor(35));
        System.out.println("Ceiling of 35: " + set.ceiling(35));
        System.out.println("Lower than 30: " + set.lower(30));
        System.out.println("Higher than 30: " + set.higher(30));
        set.remove(20);
    }
}

```



```
System.out.println("After remove(): " + set);
System.out.println("Is Empty? " + set.isEmpty());
set.clear();
System.out.println("After clear(): " + set);
}
}
```

OUTPUT:

After add(): [10, 20, 30, 40, 50]

Contains 20? true

Size: 5

First Element: 10

Last Element: 50

Floor of 35: 30

Ceiling of 35: 40

Lower than 30: 20

Higher than 30: 40

After remove(): [10, 30, 40, 50]

Is Empty? false

After clear(): []